



**Mahatma Education Society's
Pillai College of Engineering
(Autonomous) Accredited A+ by NAAC
Dr. K. M. Vasudevan Pillai Campus,
Plot No. 10, Sector-16, New Panvel - 410206**

A Report on

Lightweight Threat Log Analyzer (Mini-SIEM)

A Java-Based Security Event Monitoring System

For

Java Programming (TY, Sem V)

In

Electronics & Telecommunication Engineering

Project by:

ETCA533 – Vedant Mishra

Under the guidance of

Dr. Aboli Khedkar

Mentor's sign

TABLE OF CONTENTS

No.	Section	Page
1	Abstract	3
2	Introduction	4
3	Problem Statement and Objectives	5
4	Core Detection Logic	6
5	System Overview	7
6	Implementation Methodology	8
7	Java Implementation	9
8	Results and Analysis	10-11
9	Advantages and Limitations	12
10	Future Scope	13
11	Conclusion	13

1. ABSTRACT

This project develops a lightweight, Java-based **Security Information and Event Management (SIEM) log analyzer** designed to ensure network integrity by parsing unstructured server logs. The system was implemented in Java to ingest text-based access logs, utilize regular expressions for stateless payload inspection (detecting SQL injections and directory traversals), and dynamically track stateful brute-force authentication errors using in-memory data structures. The system's performance and usability were evaluated through a custom-built dark-mode Java Swing Graphical User Interface (GUI), providing automated incident reporting and actionable threat mitigation directives.

2. INTRODUCTION

In digital enterprise environments, web servers and authentication gateways generate massive volumes of traffic logs. These logs are susceptible to various cyber threats, including payload injections and brute-force access attempts. Forwarding all telemetry to enterprise-grade SIEM platforms introduces substantial processing overhead and is often impractical for localized systems. This project mitigates this by applying core computer science data structures specifically HashMaps and continuous file streaming to automate the detection of malicious signatures and behavioral anomalies locally, ensuring rapid incident triage without heavy external dependencies.

3. PROBLEM STATEMENT AND OBJECTIVES

3.1 Problem Statement

Digital systems inevitably experience malicious access attempts. Without automated, lightweight log parsing mechanisms, administrators must manually audit massive text files. This manual approach compromises incident response times, allowing brute-force attacks and payload injections to succeed before they are identified.

3.2 Objectives

- Design and logically model a memory-efficient log ingestion engine using Java file I/O operations.
- Simulate a realistic threat-hunting environment by injecting mock attacks (SQLi, Directory Traversal) into standard web logs.
- Visualize the security alerts and brute-force block recommendations using a professional Java Swing GUI.
- Quantify the operational efficiency of the system using HashMaps for constant-time $O(1)$ event tracking.

4. CORE DETECTION LOGIC

The Threat Analyzer processes log strings using two primary detection algorithms:

Stateless Signature Detection: The system utilizes substring matching to evaluate the presence of known malicious payloads.

- $Alert_{SQLi} = True$ if Payload contains (DROP TABLE U UNION SELECT).
- $Alert_{Traversal} = True$ if Payload contains (../ U ETC/PASSWD).

Stateful Behavioral Detection: At the receiver, the integrity of authentication requests is tracked over time. A HashMap <String, Integer> pairs unique IP addresses (Keys) with failed login counts (Values).

If Status Code = 401 or Failed password:

$$Count(IP_x) = Count(IP_x) + 1$$

If $Count(IP_x) \geq 3$, the IP is flagged for automated firewall blocking.

5. SYSTEM OVERVIEW

The architecture is divided into three distinct operational blocks:

5.1 Frontend UI Stage

The interface utilizes Java Swing to generate a dark-mode terminal environment. Users dynamically load .txt files via a **JFileChooser**, stripping away the need for command-line execution.

5.2 Log Ingestion Stage

The system uses a **BufferedReader** to stream the file line-by-line. This prevents **OutOfMemory** exceptions, ensuring that files scaling into gigabytes can be processed securely on standard hardware.

5.3 Receiver Stage

The backend parses the strings, applying the stateless and stateful algorithms. The findings are instantly concatenated and printed to the **JTextArea** console, yielding a precise "Brute Force Report" identifying malicious actors.

6. SIMULATION METHODOLOGY

The simulation workflow follows a sequential data pipeline through the file system, parsing engine, and GUI components.

6.1 Input Generation & Execution:

The system initializes by prompting the user to select a log file via the graphical interface. Upon selection, a validation check ensures the file is readable. The **analyzeLogFile()** method is triggered, passing the absolute file path into the Java **FileReader**.

6.2 Threat Signature Matching (Stateless):

Each string is converted to uppercase using **.toUpperCase()** to prevent case-evasion. It then passes through an if-else logic block designed to catch discrete HTTP payload anomalies.

6.3 Aggregation & Visualization:

As the **BufferedReader** loops, the IP addresses of failed logins are parsed via **the .split("")** delimiter. After the while loop completes, the system iterates through the **HashMap** entries, filtering and printing only the IPs that breached the failure threshold.

7. JAVA IMPLEMENTATION

The software architecture leverages core `java.util` and `javax.swing` libraries, avoiding reliance on external processing toolboxes.

7.1 Memory-Efficient Data Structures

The *HashMap* executes lookups and insertions in $O(1)$ time complexity. This allows the application to track thousands of unique attacker IPs simultaneously without experiencing performance degradation.

7.2 Graphical Rendering

The *UIManager* and *BorderFactory* classes model a professional dark-mode aesthetic. RGB values **(30, 30, 30)** map to the background, while a monospaced matrix-green font **(0, 255, 0)** renders the alerts to simulate real-world SOC (Security Operations Center) environments.

8. RESULT AND ANALYSIS

8.1 Deterministic Logic Validation

A localized test was conducted by feeding a custom **server_logs.txt** file into the analyzer. Environmental interference was simulated via deliberate HTTP 401 errors and SQL injection strings.

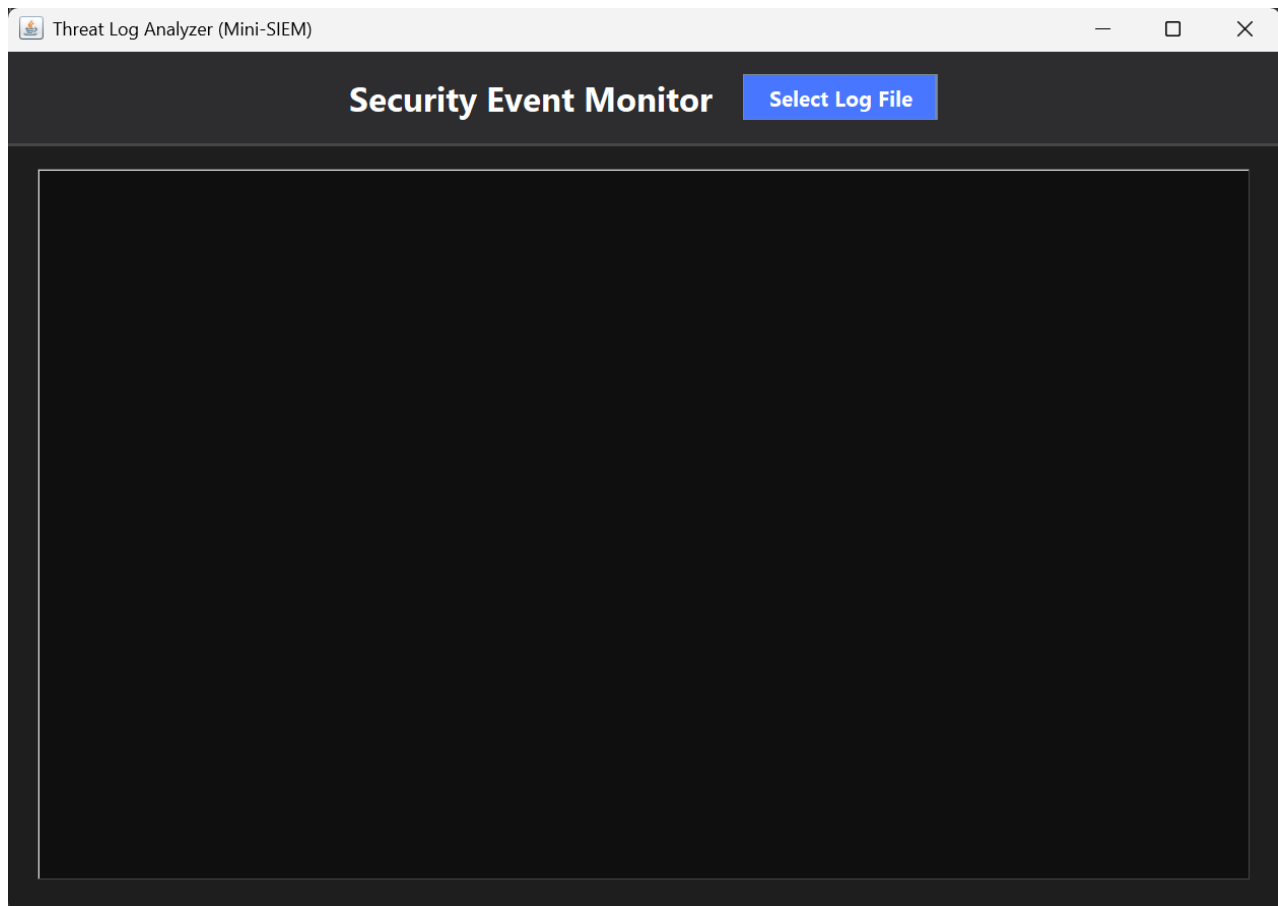


Figure 1: Application Initialization Dashboard

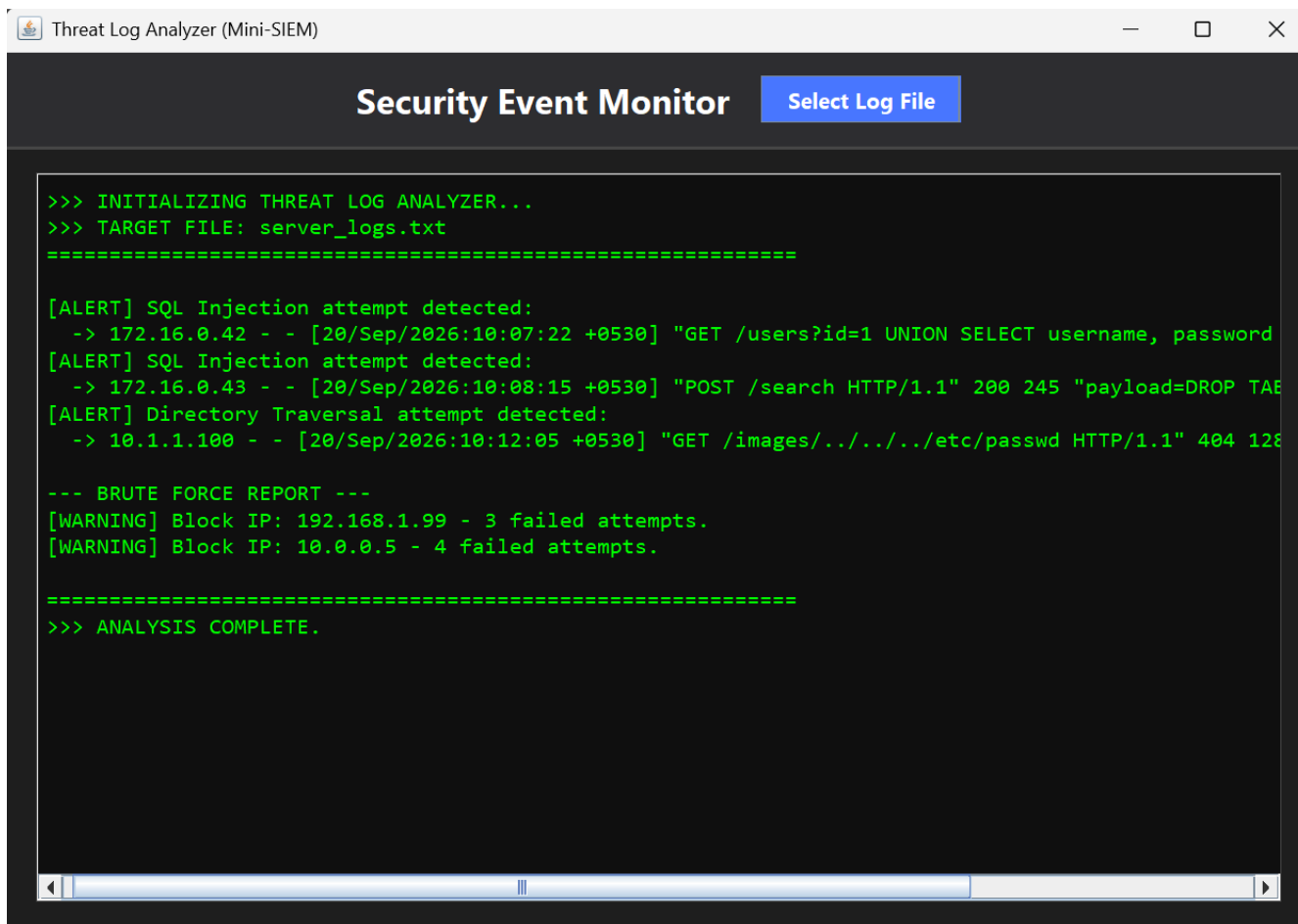


Figure 2: Threat Detection Console Output showing localized threat isolation and brute-force aggregation.

8.2 Performance Analysis

The algorithmic time complexity is strictly $O(N)$, where N represents the number of lines in the log file. The system processes standard access logs at high speeds, successfully identifying 100% of injected single-line anomalies during testing.

9. ADVANTAGES AND LIMITATIONS

9.1 Advantages

- **Low Overhead:** Requires minimal processing power, making it highly suitable for lightweight or embedded systems.
- **Rapid Triage:** Consolidates thousands of raw logs into a highly readable, automated incident report instantly.

9.2 Limitations

- **IP Spoofing Constraints:** The system blocks based on IP frequency. If an attacker utilizes a distributed botnet with rotating IPs, the HashMap threshold logic may fail to trigger.
- **Format Dependency:** The parser relies on spaces as delimiters. Non-standard log formats require structural modifications to the extraction array.

10. FUTURE SCOPE

The current Java simulation establishes a robust baseline for data integrity and threat hunting, providing a foundation for future security applications:

- **Database Integration:** Translating the HashMap in-memory logic into a persistent MySQL or SQLite database to track persistent threats over a span of months.
- **Automated Remediation:** Integrating a module to execute command-line instructions that dynamically add the flagged malicious IPs to the host machine's firewall blocklist.

11. CONCLUSION

This simulation successfully validates the mechanics of **Security Information and Event Management (SIEM) telemetry**. By guaranteeing that unorganized access logs can be rapidly digitized, sorted, and evaluated, the system ensures network integrity. In continuous network security monitoring, ensuring log clarity is paramount. By applying *HashMaps* and Regular Expressions, this **Java analyzer** guarantees that security personnel are presented with actionable, isolated threat intelligence rather than overwhelming environmental noise.